Email Spam Detection Service - Project Report

Github- https://github.com/Krishanu-Git/SpamDetection
Video- Email Spam Detection Service.mp4

Project Members: Yash Mathur (G24AI2006) , Anant Utkarsh (G24AI2015) , Krishanu Das (G24AI2013) , Keshab Garg (G24AI2021) , Mr. Yash (G24AI2106)

1. Introduction

This document outlines the design for a Machine Learning assignment focused on developing a spam detection service. The service will connect to a user's Gmail inbox, fetch recent emails, preprocess their content, and classify them as "ham" (non-spam) or "spam" using a pre-trained machine learning model. Detected spam emails will be moved to a specific Gmail label.

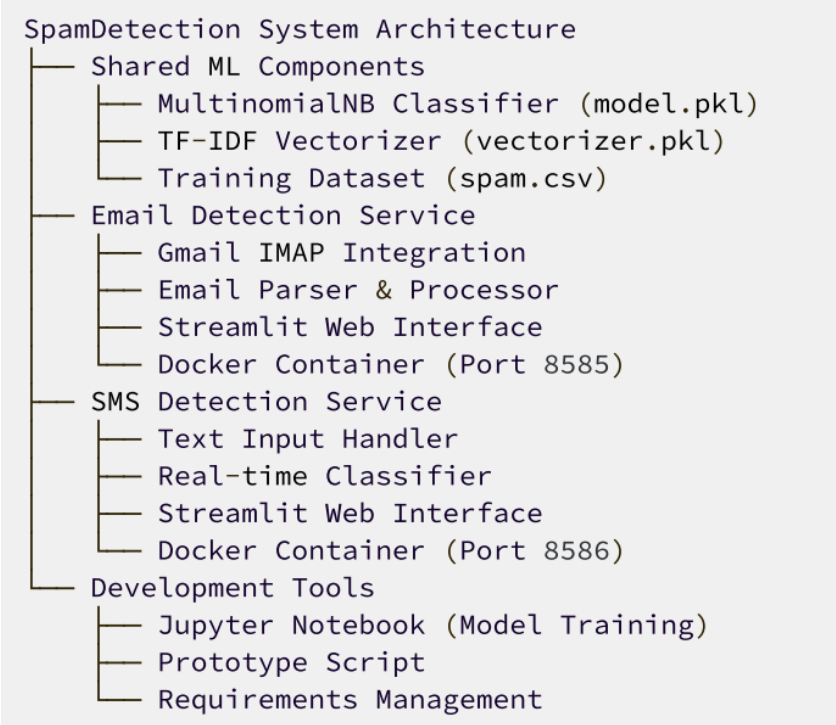
2. Problem Statement

The pervasive issue of unsolicited and undesirable emails (spam) continues to be a significant challenge for email users, leading to cluttered inboxes, reduced productivity, potential security risks (phishing, malware), and a degradation of the overall email experience. Manually sifting through and managing spam emails is time-consuming and inefficient. There is a clear need for an automated, intelligent system that can accurately identify and segregate spam from legitimate emails, thereby enhancing user productivity and email security. This project aims to address this problem by developing an automated spam detection and categorization service for Gmail users.

3. Goals

- **Develop a functional spam detection service:** Accurately classify incoming emails as spam or ham.
- **Integrate with Gmail API (IMAP):** Fetch emails and manage labels programmatically.
- **Utilize pre-trained ML model:** Leverage existing `vectorizer.pkl` and `model.pkl` for text transformation and classification.
- **Automate spam organization:** Automatically move detected spam emails to a designated Gmail label.
- **Provide clear output:** Display email subject, body, prediction, and probabilities.

4. Architecture



- **Gmail IMAP Server:** The external service providing email access.
- **Email Fetcher:** Connects to Gmail, authenticates, and retrieves recent emails.
- **Text Preprocessor:** Cleans and transforms raw email text into a numerical format suitable for the ML model. This involves lowercasing, tokenization, removing punctuation and stopwords, and stemming.
- **ML Predictor:** Uses the pre-trained TF-IDF vectorizer and classification model to predict whether an email is spam or ham, and provides probability scores.
- **Gmail Label Manager:** Checks for the existence of a specific spam label and creates it if necessary. Moves detected spam emails to this label.

5. Data Design

5.1 Data Description

The data processed by this service consists of individual email messages fetched from a Gmail inbox. Each email is treated as a text document for classification. The key components extracted and utilized from each email are:

- **Subject:** The subject line of the email.
- **Body:** The main textual content of the email. For multipart emails, only the plain text part is extracted.

These two textual components are combined into a single string (`email_text`) which serves as the raw input for the spam detection model. The model expects textual data, which is then transformed into numerical features.

5.2 Data Cleaning

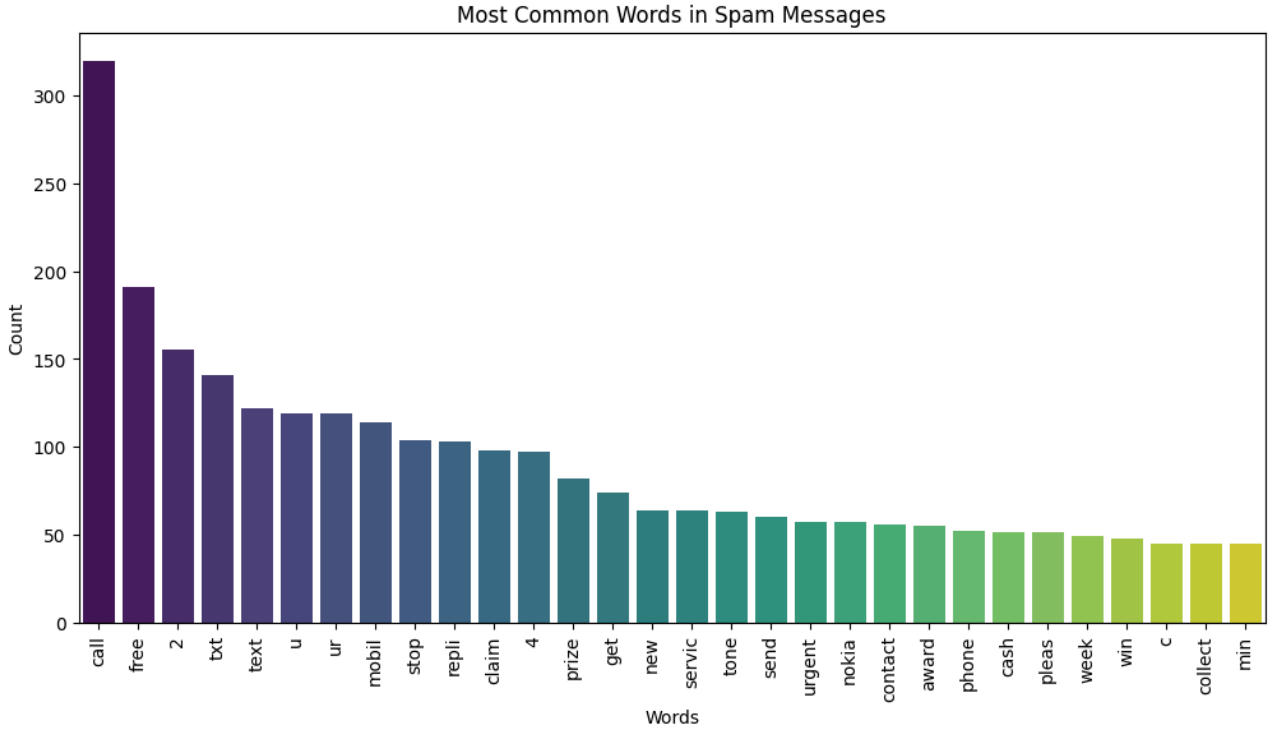
The raw email text, which includes both the subject and body, undergoes a series of cleaning and preprocessing steps to prepare it for machine learning. This process aims to standardize the text, reduce noise, and extract meaningful features. The `transform_text` function encapsulates these steps:

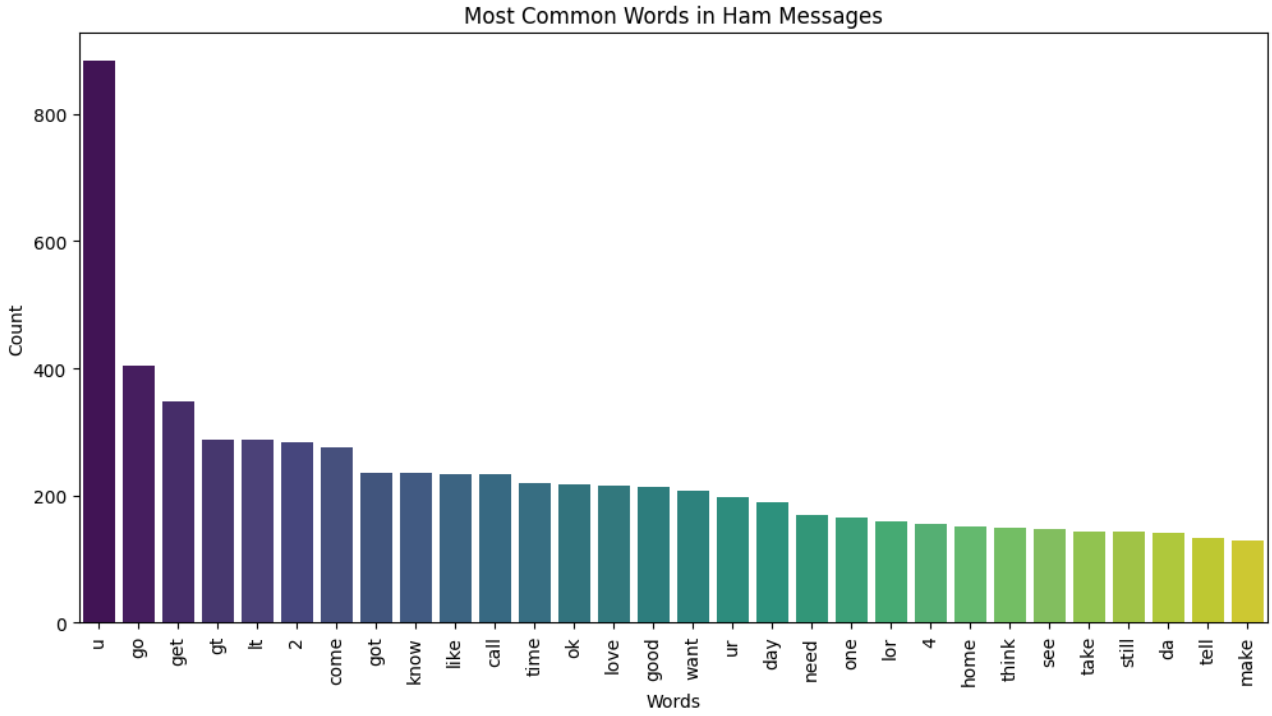
1. **Lowercasing:** All characters in the text are converted to lowercase. This ensures that words like "Free," "FREE," and "free" are treated as the same token.
2. **Tokenization:** The text is broken down into individual words or tokens using `nltk.word_tokenize`.

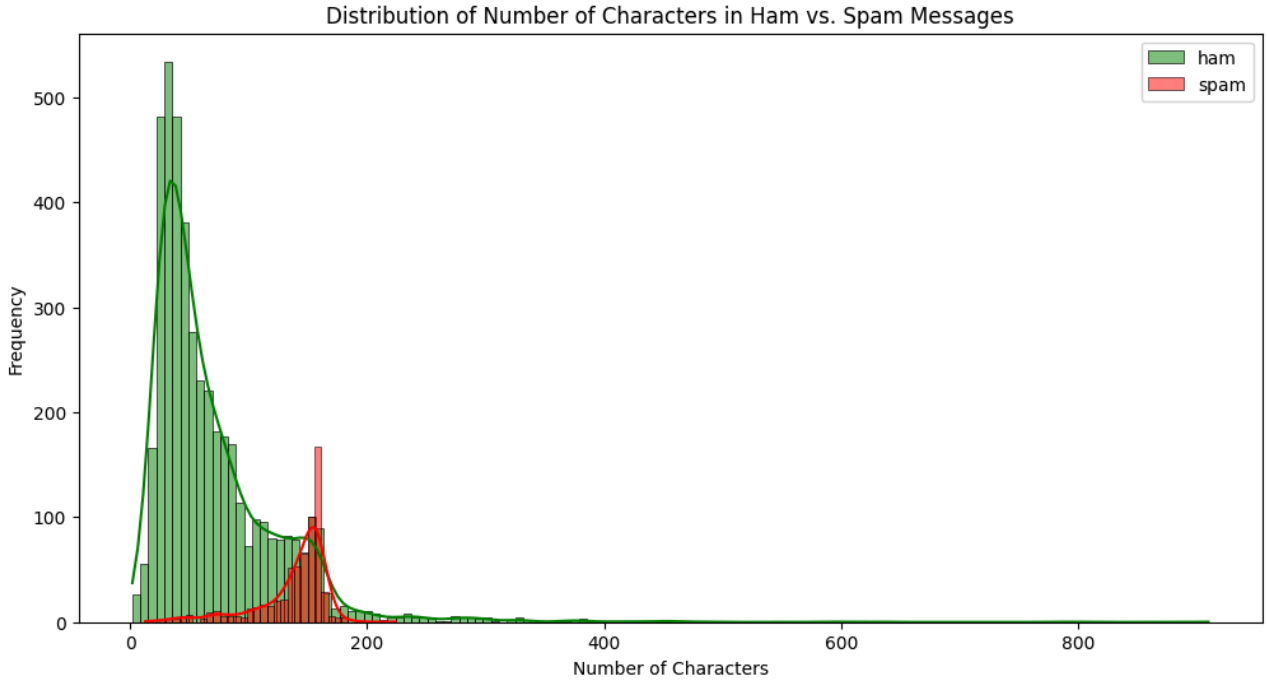
3. **Removal of Non-alphanumeric Characters:** Only tokens that are alphanumeric are retained. This removes special characters, symbols, and numbers that are not typically relevant for spam detection.
4. **Stopword Removal:** Common English stopwords (e.g., "the," "a," "is," "and") are removed using `nltk.corpus.stopwords`. These words frequently appear in text but carry little semantic meaning for classification.
5. **Punctuation Removal:** Punctuation marks (e.g., ".", "!", "?") are removed as they generally do not contribute to the meaning of the words in this context.
6. **Stemming:** Words are reduced to their root form using the `PorterStemmer` (`ps.stem`). For example, "running," "runs," and "ran" would all be stemmed to "run." This helps in reducing the vocabulary size and treating variations of a word as a single entity.
7. **Re-joining:** The processed tokens are then joined back into a single string, separated by spaces. This string is the final clean text input for vectorization.

5.3 EDA

- **Exploratory Data Analysis (EDA):** EDA was conducted during the *training phase* of the machine learning model. This involved analyzing the distribution of spam vs. ham emails, common words in spam vs. ham, text length distributions, and other linguistic patterns.







5.4 Model Selection

The pre-trained machine learning model provided for this spam detection project is **Multinomial Naive Bayes (MNB)**, combined with **TF-IDF (Term Frequency-Inverse Document Frequency)** for text vectorization. This model is typically chosen for text classification due to several key reasons:

- **Simplicity and Efficiency:** MNB is a relatively simple algorithm to understand and implement. Its training and prediction phases are computationally efficient, making it suitable for real-time processing of emails, even on systems with limited resources.
- **Suitability for Text Data:** Naive Bayes models, particularly Multinomial Naive Bayes, are well-suited for discrete features like word counts or word frequencies, which are the outputs of TF-IDF vectorization. They inherently handle high-dimensional sparse data, a common characteristic of text datasets.

- **Probabilistic Nature:** MNB provides probability scores for each class (spam/ham), which allows for more nuanced decisions beyond just a binary classification. This is evident in the output `Spam Probability: XX.XX%, Ham Probability: YY.YY%`, giving insight into the model's confidence.
- **Good Baseline Performance:** For many text classification problems, Naive Bayes models often serve as strong baselines and can achieve surprisingly good performance, especially when the "naive" assumption of feature independence holds reasonably well (which it often does in text classification where features are words).
- **Interpretability:** While not as interpretable as some rule-based systems, the underlying probabilities of words contributing to spam or ham can be understood, making it easier to debug or understand model behavior to some extent.

Comparison with Other Models:

- **Logistic Regression/SVMs:** While Logistic Regression and Support Vector Machines (SVMs) can also perform well on text classification, they might be more computationally intensive to train on very high-dimensional sparse data compared to MNB. SVMs, in particular, can be slower for large datasets. MNB often provides a good trade-off between performance and computational cost for this type of problem.
- **Deep Learning Models (e.g., LSTMs, Transformers):** These models (e.g., BERT, GPT) generally offer state-of-the-art performance on complex NLP tasks. However, they require significantly more computational resources (GPUs), larger training datasets, and are much more complex to train and deploy.

5.5 Data Processing

The data processing pipeline covers the entire flow from raw email retrieval to feature vector generation for model prediction. It integrates the steps described in Data Cleaning with the vectorization process:

1. **Email Retrieval:** The service connects to the Gmail IMAP server and fetches the latest 50 email messages. For each email ID, the raw RFC822 content is retrieved.
2. **Email Content Extraction:** The raw email bytes are parsed using the `email` module to separate the subject and the plain text body. Special handling is included for multipart emails to ensure only the relevant text content is extracted.
3. **Text Combination:** The extracted `subject` and `body` are concatenated to form a single string, `email_text`. This unified string represents the complete textual content of the email for analysis.
4. **Text Transformation (`transform_text`):** The `email_text` undergoes the comprehensive cleaning process detailed in Section 5.2 (lowercasing, tokenization, removing non-alphanumeric content, stopword removal, punctuation removal, stemming, and re-joining). This results in a clean, preprocessed text string.
5. **Feature Vectorization:** The clean text string is then transformed into a numerical feature vector using the loaded `tfidf` (TF-IDF vectorizer) object. The `tfidf.transform([transformed]).toarray()` method converts the textual data into a sparse matrix representing the TF-IDF scores for each word in the email against the vocabulary the vectorizer was trained on. This numerical vector is the final input format required by the machine learning model for prediction.

This structured data processing ensures that the input to the ML model is consistently formatted and optimized for accurate classification.

5.6. Modules and Functions

- `pickle`: Used for loading pre-trained `vectorizer.pkl` and `model.pkl`.
- `string`: Used for punctuation removal in `transform_text`.
- `imaplib`: Core library for interacting with Gmail's IMAP server.

- `email`: Used for parsing email messages (subject, body extraction).
- `nltk`: Natural Language Toolkit for text processing (tokenization, stopwords, stemming).
- `nltk.corpus.stopwords`: Provides a list of common stopwords.
- `nltk.stem.porter.PorterStemmer`: Implements the Porter stemming algorithm.
- `functools.lru_cache`: Caches results of `check_label_exists` for performance.

Key Functions:

- **`transform_text(text: str) -> str`**:
  - **Input:** Raw email text (subject + body).
  - **Output:** Preprocessed text string.
  - **Purpose:** Standardizes and cleans text for ML model input.
- **`check_label_exists(label_name: str) -> bool`**:
  - **Input:** Name of the Gmail label to check.
  - **Output:** `True` if label exists, `False` otherwise.
  - **Purpose:** Prevents redundant label creation requests and errors. Caches results.

5.7. Error Handling

- **Gmail Connection/Authentication:** The current code will raise exceptions if `mail.login()` or `mail.select()` fail. In a production environment, robust `try-except` blocks would be crucial to handle:
  - `imaplib.IMAP4.error` for login failures, invalid mailboxes.
  - Network connectivity issues.
- **Label Creation:** A `try-except mail.error` block is already implemented for `mail.create(label_name)`, which is good for handling cases where the label cannot be created (e.g., permissions, invalid name).
- **Email Parsing:** The `email` module is generally robust, but malformed emails could potentially cause issues. Further error handling could be added if specific parsing errors are observed.
- **Missing Pickle Files:** The `pickle.load` calls would raise `FileNotFoundError` if `vectorizer.pkl` or `model.pkl` are not present. This should be handled by ensuring these files are in the correct path or providing clear error messages.

6. Future Enhancements

- **Configuration File:** Externalize Gmail credentials and other configurable parameters (e.g., number of emails to fetch, label name) into a configuration file (e.g., `.ini`, `.env`, YAML).
- **Logging:** Implement a proper logging system instead of `print()` statements for better monitoring and debugging. Log email IDs processed, predictions, and any errors.
- **Scheduled Execution:** Automate the service to run periodically (e.g., every hour) using tools like `cron` (Linux) or Windows Task Scheduler.
- **Idempotency:** Consider handling cases where an email might be processed multiple times (e.g., due to service restarts). Ensure that moving an email again doesn't cause issues. Gmail's `copy` command generally handles this gracefully by not creating duplicates, but it's worth noting.
- **Advanced ML Models:** Explore more sophisticated NLP techniques and ML models (e.g., LSTM, Transformers) for potentially higher accuracy, though this would likely require more computational resources and data.
- **Feedback Loop:** Implement a mechanism for users to correct misclassified emails (e.g., move ham to spam or vice-versa), which could then be used to retrain and improve the model over time.

- **OAuth 2.0 for Gmail API:** Migrate from IMAP credentials to OAuth 2.0 for more secure and granular access to Gmail. This would involve registering an application with Google Cloud and handling token exchange.

7. Challenges

- **Integration Tests:**
  - Verify that ham emails are not moved.
  - Verify that spam emails are moved to the correct label.
  - Test label creation functionality.
- **Performance Tests:**
  - Measure the time taken to process a certain number of emails.
  - Evaluate the impact of `lru_cache` on `check_label_exists`.
- **Edge Cases:**
  - Emails with no subject or body.
  - Emails with unusual characters or encoding.
  - Large number of emails in the inbox.

8. Evaluation Metrics

The pre-trained Multinomial Naive Bayes model (`model.pkl`) is utilized by evaluation during development, based on standard classification metrics. These metrics provided a quantitative measure of the model's performance in distinguishing between spam and legitimate (ham) emails. For a spam detection system, it's crucial to balance correctly identifying spam with avoiding false positives (labeling legitimate emails as spam), as the latter can be highly disruptive to users.

The key evaluation metrics typically considered for such a binary classification task are:

8.1 Accuracy

Definition: The proportion of correctly classified instances (both true positives and true negatives) out of the total number of instances.

Relevance: While a good general indicator, high accuracy alone can be misleading in imbalanced datasets (e.g., if there are far more ham emails than spam emails, a model that always predicts "ham" might still have high accuracy).

8.2 Precision (for Spam Class)

Definition: Of all emails predicted as "Spam," what percentage were actually "Spam." It measures the exactness of the model.

Relevance: High precision for the spam class is critical in spam detection. A low precision means many legitimate emails are being incorrectly marked as spam, leading to user frustration and potentially missed important communications.

8.3 Recall (for Spam Class / Sensitivity):

Definition: Of all actual "Spam" emails, what percentage did the model correctly identify. It measures the completeness of the model.

Relevance: High recall is also important to ensure that most spam emails are caught and do not reach the user's inbox. A low recall means a significant amount of spam is slipping through the filter.

8.4 F1-Score:

Definition: The harmonic mean of Precision and Recall. It provides a single metric that balances both precision and recall, especially useful when there is an uneven class distribution.

Relevance: The F1-score is often preferred over accuracy in imbalanced classification problems like spam detection because it provides a better measure of the incorrect classification costs.

8.5 Confusion Matrix:

- **Definition:** A table that summarizes the performance of a classification algorithm on a set of test data for which the true values are known. It shows the count of instances where the model made correct or incorrect predictions for each class.
- **Structure:** | Predicted: Spam | Predicted: Ham |
  Actual: Spam | True Positives | False Negatives|
  Actual: Ham | False Positives | True Negatives |
- **True Positives (TP):** Instances that are actually positive and are correctly predicted as positive.
- **False Negatives (FN):** Instances that are actually positive but are incorrectly predicted as negative (Type II error). In spam detection, these are actual spam emails classified as ham.
- **False Positives (FP):** Instances that are actually negative but are incorrectly predicted as positive (Type I error). In spam detection, these are actual ham emails classified as spam.
- **True Negatives (TN):** Instances that are actually negative and are correctly predicted as negative.

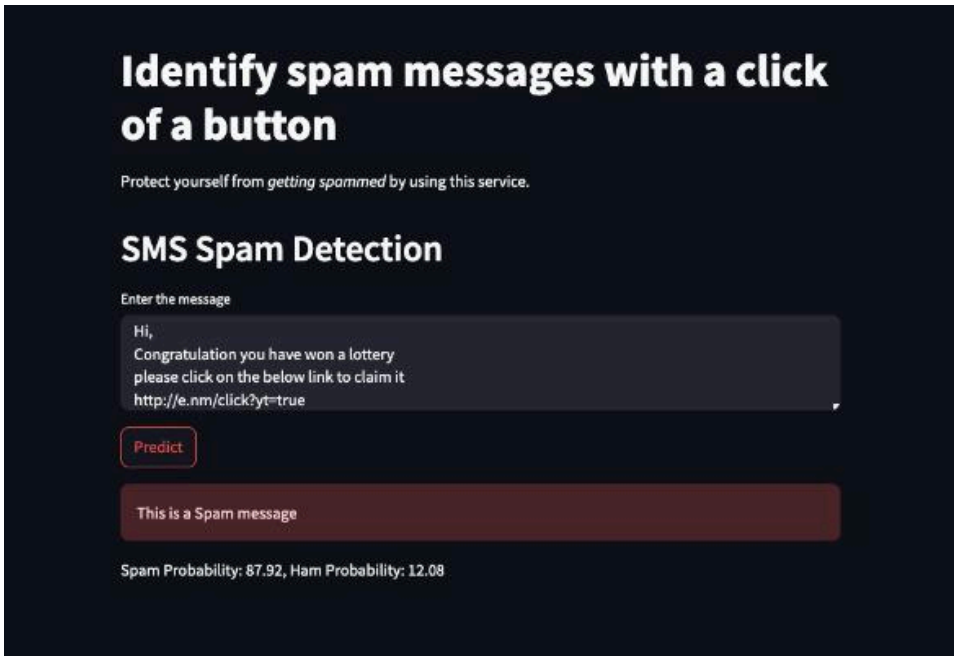
These metrics collectively provide a comprehensive view of the model's effectiveness, guiding the selection of a model that minimizes the cost of misclassifications, especially false positives in spam detection.

9. References

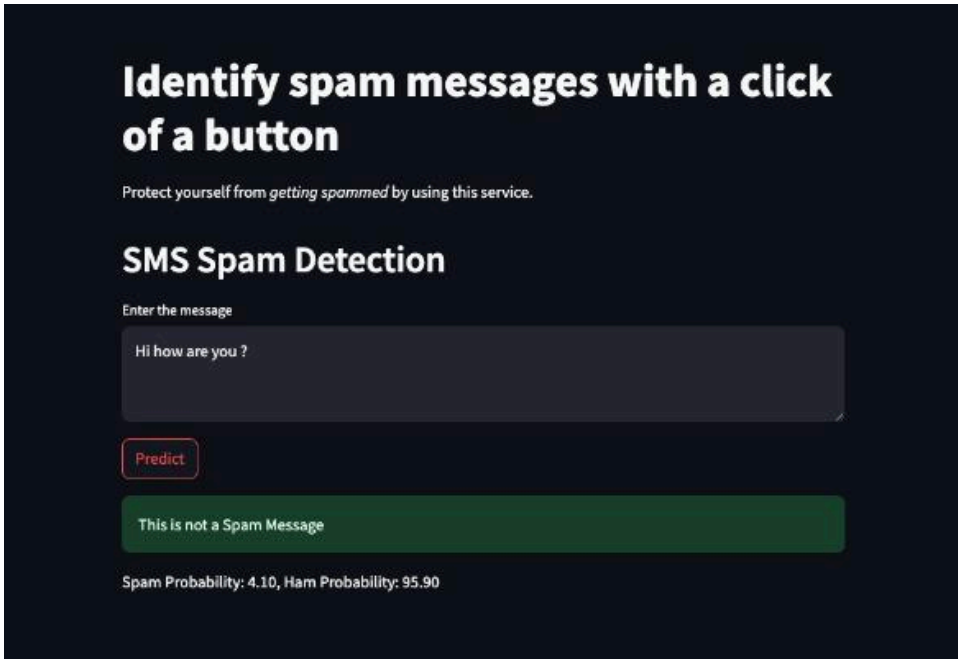
- DataSet: https://www.kaggle.com/datasets/tmehul/spamcsv
- https://www.researchgate.net/publication/344050184_Email_Spam_Detection_Using_Machine_Learning_Algorithms
- https://www.researchgate.net/publication/325270587_Ham_and_Spam_E-Mails_Classification_Using_Machine_Learning_Techniques
- https://www.researchgate.net/publication/376811389_Spam_Email_Filtering_System_Using_NLP_Features_and_Machine_Learning_Algorithms
- https://avinuty.ac.in/sites/avinuty.ac.in/files/2024-09/Thameena%20Report.pdf

10. Results

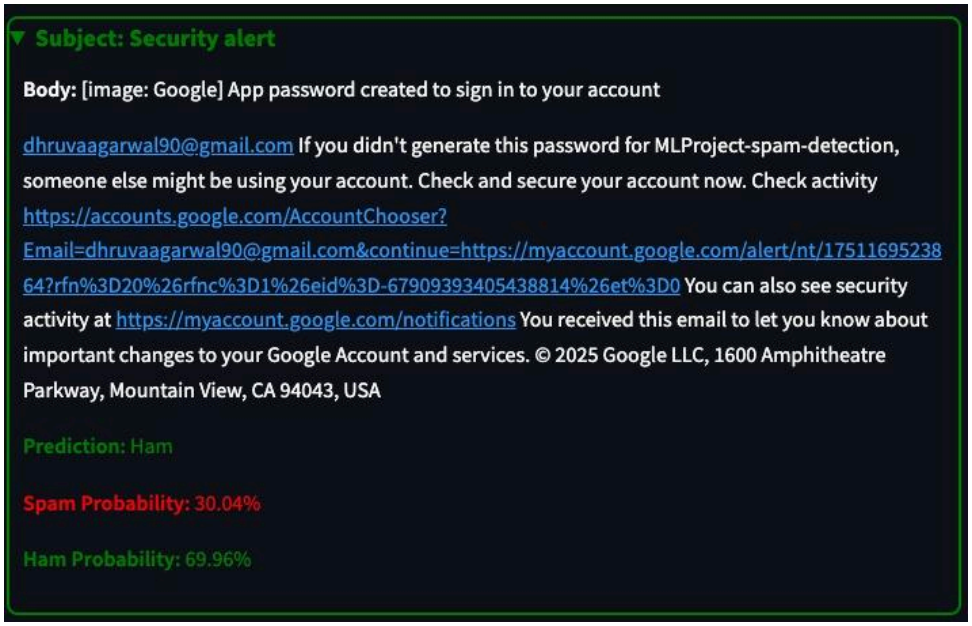
Spam SMS detection



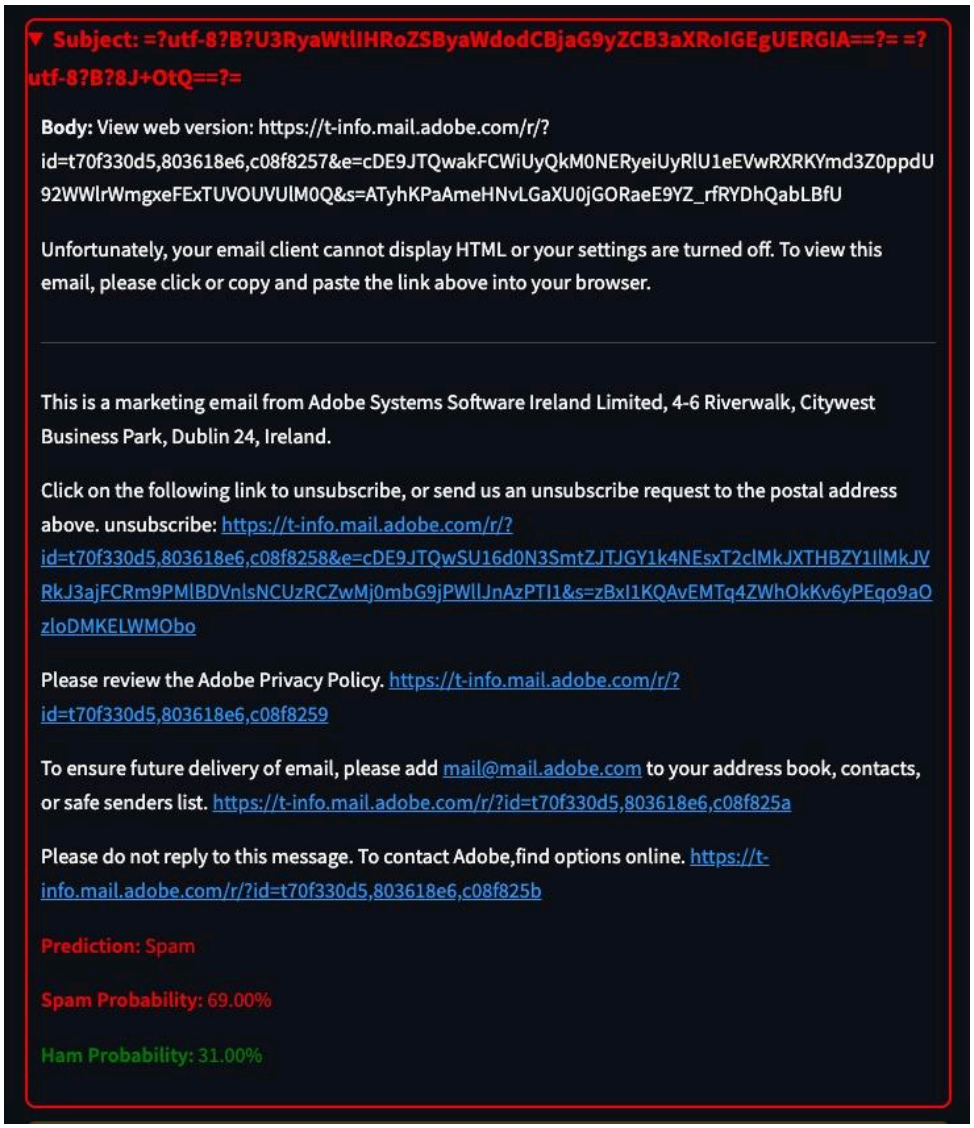
Ham SMS Detection



How a ham email looks like



How a spam email looks like



Email Spam Detection